

Projekt z uczenia maszynowego

# Nauczenie komputera sterowania samochodem w grze 2D

Sieć neuronowa jako kierowca pojazdu sterowanego  
na podstawie sygnałów z czujników odległości

Raport końcowy

5 czerwca 2026

## Spis treści

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>1</b>  | <b>Cel projektu</b>                                  | <b>3</b>  |
| <b>2</b>  | <b>Wykorzystany model uczenia maszynowego</b>        | <b>3</b>  |
| 2.1       | Idea modelu . . . . .                                | 3         |
| 2.2       | Uzasadnienie wyboru . . . . .                        | 4         |
| <b>3</b>  | <b>Opis środowiska — gra w C++</b>                   | <b>4</b>  |
| 3.1       | Model fizyki pojazdu . . . . .                       | 4         |
| 3.2       | Czujniki — „oczy” kierowcy . . . . .                 | 5         |
| 3.3       | Zbieranie danych . . . . .                           | 6         |
| <b>4</b>  | <b>Sformułowanie problemu uczenia maszynowego</b>    | <b>6</b>  |
| 4.1       | Cechy wejściowe . . . . .                            | 6         |
| 4.2       | Funkcja straty i nierównowaga klas . . . . .         | 7         |
| <b>5</b>  | <b>Dane i ich przygotowanie</b>                      | <b>8</b>  |
| 5.1       | Charakterystyka zbioru . . . . .                     | 8         |
| 5.2       | Kroki przygotowania danych . . . . .                 | 8         |
| 5.3       | Spójność uczenia i wdrożenia . . . . .               | 9         |
| <b>6</b>  | <b>Analiza eksploracyjna danych</b>                  | <b>10</b> |
| 6.1       | Rozkład etykiet i nierównowaga klas . . . . .        | 10        |
| 6.2       | Wnioski z analizy zbalansowania . . . . .            | 10        |
| <b>7</b>  | <b>Architektura modelu</b>                           | <b>10</b> |
| 7.1       | Teoria użytych mechanizmów . . . . .                 | 11        |
| <b>8</b>  | <b>Metodyka eksperymentów</b>                        | <b>12</b> |
| 8.1       | Podział danych i powtarzalność . . . . .             | 12        |
| 8.2       | Procedura treningu . . . . .                         | 12        |
| 8.3       | Metryki oceny . . . . .                              | 13        |
| 8.4       | Logika decyzyjna i progi . . . . .                   | 14        |
| <b>9</b>  | <b>Model bazowy i strojenie konfiguracji</b>         | <b>14</b> |
| 9.1       | Model bazowy . . . . .                               | 14        |
| 9.2       | Przeszukiwanie przestrzeni hiperparametrów . . . . . | 14        |
| <b>10</b> | <b>Wyniki</b>                                        | <b>15</b> |
| 10.1      | Przebieg uczenia modelu bazowego . . . . .           | 15        |
| 10.2      | Jakość poszczególnych akcji . . . . .                | 16        |
| 10.3      | Wyniki strojenia hiperparametrów . . . . .           | 17        |
| 10.4      | Porównanie z modelem bazowym . . . . .               | 18        |
| <b>11</b> | <b>Wdrożenie modelu w grze</b>                       | <b>18</b> |

|                                            |           |
|--------------------------------------------|-----------|
| <b>12 Analiza błędów i ograniczenia</b>    | <b>19</b> |
| <b>13 Wnioski i kierunki dalszych prac</b> | <b>20</b> |

## 1 Cel projektu

Celem projektu jest **nauczenie komputera samodzielnego sterowania samochodem w grze 2D**. Zadaniem wytrenowanego modelu jest obserwowanie stanu pojazdu i jego otoczenia, a następnie podejmowanie w czasie rzeczywistym decyzji sterujących (przyspieszanie, hamowanie, skręt w lewo, skręt w prawo) tak, aby pojazd poruszał się po torze i dojechał do mety, nie uderzając w przeszkody.

Należy podkreślić, że *właściwym celem projektu nie jest samo testowanie wartości hiperparametrów*. Strojenie konfiguracji modelu (rozdział 9) jest jedynie **środkiem** prowadzącym do celu nadrzędnego — uzyskania modelu, który skutecznie zastępuje człowieka przy kierownicy. Eksperymenty z hiperparametrami traktujemy więc jako narzędzie poprawiające jakość kierowcy, a nie jako produkt sam w sobie.

Projekt obejmuje pełny proces uczenia maszynowego:

1. zebranie danych z gry (rozgrywka człowieka jako wzorzec),
2. przygotowanie i normalizację danych,
3. analizę eksploracyjną zebranego zbioru,
4. zaprojektowanie i wytrenowanie sieci neuronowej,
5. wyznaczenie modelu bazowego i jego optymalizację,
6. ocenę modelu metrykami klasyfikacji,
7. wdrożenie modelu w grze i sterowanie pojazdem w czasie rzeczywistym.

## 2 Wykorzystany model uczenia maszynowego

W projekcie kierowcę-komputer realizuje **sztuczna sieć neuronowa typu wielowarstwowy perceptron** (ang. *multilayer perceptron*, MLP; sieć jednokierunkowa, *feedforward*). Jest to model uczenia nadzorowanego, który odwzorowuje wektor cech opisujący sytuację na torze na wektor decyzji sterujących.

### 2.1 Idea modelu

Sieć złożona jest z kolejnych *warstw gęstych* (w pełni połączonych). Każdy neuron warstwy oblicza ważoną sumę swoich wejść powiększoną o wyraz wolny (*bias*), a wynik przepuszcza przez nieliniową funkcję aktywacji. Dla  $l$ -tej warstwy:

$$\mathbf{h}^{(l)} = \phi\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad \mathbf{h}^{(0)} = \mathbf{x}, \quad (1)$$

gdzie  $\mathbf{x} \in \mathbb{R}^{81}$  to wektor cech wejściowych (rozdział 4),  $\mathbf{W}^{(l)}$  i  $\mathbf{b}^{(l)}$  to wagi i biasy uczone podczas treningu, a  $\phi$  to funkcja aktywacji (w naszym przypadku ReLU). Warstwa wyjściowa jest *liniowa* (bez aktywacji) i zwraca

cztery *logity*  $\mathbf{z} = (z_{\text{acc}}, z_{\text{brake}}, z_{\text{left}}, z_{\text{right}}) \in \mathbb{R}^4$ , zamieniane na prawdopodobieństwa funkcją sigmoidalną (rozdział 4.2). Uczenie polega na takim doborze wag  $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$  (metodą spadku gradientu i propagacji wstecznej), aby minimalizować funkcję straty.

## 2.2 Uzasadnienie wyboru

MLP dobrze pasuje do tego zadania, ponieważ:

- wejście ma **stałą długość** (81 cech) i postać tabelaryczną, bez struktury przestrzennej czy sekwencyjnej, która uzasadniałaby sieci spłotowe (CNN) lub rekurencyjne (RNN);
- wnioskowanie jest **bardzo szybkie**, co jest kluczowe przy sterowaniu w czasie rzeczywistym (predykcja w każdej klatce gry);
- model ma wystarczającą pojemność, by uchwycić nieliniowe zależności między odczytami czujników a decyzjami kierowcy.

Szczegółową architekturę konkretnej sieci (**DriverNet**) opisano w rozdziale 7, a teorię użytych w niej mechanizmów — w rozdziale 7.1.

## 3 Opis środowiska — gra w C++

Środowiskiem projektu jest autorska gra napisana w języku C++ z wykorzystaniem biblioteki graficznej **raylib**. Gra przedstawia jazdę samochodem w widoku z góry (2D). Gracz buduje tor z klocków (linie barierowe, filary, zakręty oraz bloki typu start/meta/checkpoint), a następnie steruje pojazdem.

### 3.1 Model fizyki pojazdu

Pojazd opisany jest położeniem, kątem obrotu oraz prędkością skalarną. Sterowanie odbywa się czterema niezależnymi sygnałami (*Controls*): **accelerate**, **brake**, **steerLeft**, **steerRight**. Uproszczona dynamika (z pliku **Car.cpp**) zakłada m.in. naturalne tłumienie prędkości, ograniczenie prędkości do przedziału  $[-5, 10]$  oraz zależność siły skrętu od prędkości:

```

1 void Car::updateSpeedRot(Controls inputs) {
2     this->speed *= 0.995f;           // naturalne
3     tlumienie
4     if (inputs.accelerate && (this->speed < 10)) this->speed
        += 0.1f;
5     if (inputs.brake && (this->speed > -5)) this->speed
        -= 0.3f;
6     if (this->speed < -5) this->speed = -5;
7     if (this->speed > 10) this->speed = 10;
8
9     float maxTurn = 3.0f, turnSpeed = 0;
10    if (abs(this->speed) < 0.5f) turnSpeed = 0;
11    else if (abs(this->speed) <= 5.0f) turnSpeed =
        (this->speed/5.0f)*maxTurn;
12    else turnSpeed = maxTurn*(1.0f -
        0.5f*((abs(this->speed)-5.0f)/5.0f));
13
14    if (inputs.steerLeft) this->rotation -= turnSpeed;
15    if (inputs.steerRight) this->rotation += turnSpeed;
16 }

```

Listing 1: Aktualizacja prędkości i obrotu pojazdu (fragment `Car.cpp`).

Kolizja z barierą (typ 0) powoduje reset pojazdu do punktu odrodzenia, blok checkpoint (typ 2) aktualizuje punkt odrodzenia, a wjazd na metę (typ 3) zatrzymuje licznik czasu i kończy przejazd.

### 3.2 Czujniki — „oczy” kierowcy

Postrzeganie otoczenia przez pojazd zrealizowano za pomocą **20 promieni (raycastów)** rozchodzących się od środka samochodu. Trzynaście promieni pokrywa przedni wachlarz (co  $10^\circ$ ), a kolejnych siedem obserwuje tył i boki. Dla każdego promienia wyznaczana jest:

- **odległość** do najbliższego trafionego obiektu (`ray_dist`),
- **typ** trafionego obiektu (`ray_type`):  $-1$  — brak trafienia,  $0$  — ściana/przeszkoda,  $1$  — meta.

Te 20 odległości oraz 20 typów wraz z prędkością tworzą pełny *stan gry* (`GameState`), który stanowi wejście modelu. Zestaw promieni jest aktualizowany względem aktualnego obrotu pojazdu:

```

1 void Car::UpdateRays() {
2     for (int i = 0; i < 13; i++)
3         this->Rays[i] = { cos((-60 + 10*i +
4                             this->rotation)*DEG2RAD),
5                             sin((-60 + 10*i +
6                                 this->rotation)*DEG2RAD) };
7     for (int i = 0; i < 7; i++)
8         this->Rays[i+13] = { cos((100 + 30*i +
9                                 this->rotation)*DEG2RAD),
10                             sin((100 + 30*i +
11                                 this->rotation)*DEG2RAD) };
12 }

```

Listing 2: Wyznaczanie kierunków promieni (fragment `Car.cpp`).

### 3.3 Zbieranie danych

Podczas jazdy sterowanej przez człowieka gra zapisuje do pliku CSV kolejne stany gry razem z wciśniętymi w danej klatce klawiszami. Nagłówek pliku generowany jest w `GameState.cpp` i ma postać:

```

car_speed, ray_dist_0, ray_type_0, ..., ray_dist_19, ray_type_19,
accelerate, brake, steer_left, steer_right.

```

W ten sposób człowiek pełni rolę „nauczyciela”: model uczy się odwzorowywać decyzje gracza w zależności od sytuacji na torze (uczenie nadzorowane, ang. *imitation learning*).

## 4 Sformułowanie problemu uczenia maszynowego

Zadanie sterowania pojazdem sprowadzono do **klasyfikacji wieloetykietowej** (ang. *multi-label classification*). Dla danego stanu  $\mathbf{x}$  model przewiduje cztery niezależne prawdopodobieństwa odpowiadające czterem klawiszom sterującym:

$$\hat{\mathbf{y}} = (p_{\text{acc}}, p_{\text{brake}}, p_{\text{left}}, p_{\text{right}}), \quad p_k \in (0, 1).$$

Etykiety nie są wzajemnie wykluczające się (np. można jednocześnie przyspieszać i skręcać), dlatego nie jest to klasyczna klasyfikacja jednoklasowa, lecz cztery sprzężone problemy binarne uczone wspólną siecią.

### 4.1 Cechy wejściowe

Wejście modelu budowane jest identycznie po stronie Pythona (uczenie) i C++ (wnioskowanie w grze). Składa się z **81 cech**:

- 1 cecha numeryczna: `car_speed`,

- 20 cech numerycznych: `ray_dist_0..19`,
- 60 cech binarnych: zakodowanie „one-hot” typu każdego z 20 promieni (3 możliwe wartości:  $-1, 0, 1$ ).

Cechy numeryczne (`car_speed` oraz odległości) są standaryzowane (rozdział 5), natomiast cechy typu promienia, jako zmienne kategoryczne, kodowane są binarnie bez skalowania.

## 4.2 Funkcja straty i nierównowaga klas

**Funkcja sigmoidalna.** Logit  $z_k$  zwracany przez sieć dla  $k$ -tej akcji zamieniany jest na prawdopodobieństwo funkcją sigmoidalną (logistyczną):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1). \quad (2)$$

Sigmoida ścisła dowolną liczbę rzeczywistą do przedziału  $(0, 1)$ , dzięki czemu  $p_k = \sigma(z_k)$  można interpretować jako prawdopodobieństwo wciśnięcia danego klawisza. Każda z czterech akcji ma własną, niezależną sigmoidę — to właśnie odróżnia klasyfikację wieloetykiętową od jednoklasowej (gdzie stosuje się *softmax* wymuszający sumę prawdopodobieństw równą 1).

**Binarna entropia krzyżowa.** Dla pojedynczej etykiety o wartości prawdziwej  $y \in \{0, 1\}$  i przewidzianym prawdopodobieństwie  $p = \sigma(z)$  funkcją straty jest binarna entropia krzyżowa (ang. *binary cross-entropy*, BCE):

$$\ell(y, p) = -[y \log p + (1 - y) \log(1 - p)]. \quad (3)$$

Strata jest mała, gdy  $p$  jest bliskie wartości  $y$ , a rośnie tym szybciej, im pewniejsza jest błędna predykcja (silne karanie pewnych pomyłek).

**Wersja stabilna numerycznie.** Zastosowana funkcja `BCEWithLogitsLoss` łączy sigmoidę (2) ze stratą (3) i liczy ją bezpośrednio z logitu  $z$ , co zapobiega niestabilności dla dużych  $|z|$ :

$$\ell(y, z) = \max(z, 0) - z y + \log(1 + e^{-|z|}). \quad (4)$$

**Ważenie klasy pozytywnej.** Ponieważ klawisze sterujące występują z bardzo różną częstością (hamowanie jest rzadkie, a przyspieszanie niemal stałe), do każdej etykiety przypisano wagę klasy pozytywnej  $w_k$  (`pos_weight`), która wzmacnia wkład rzadkich, lecz istotnych przykładów pozytywnych:

$$\ell_w(y, z) = -[w y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]. \quad (5)$$



Wagę dobiera się na podstawie proporcji liczby przykładów negatywnych do pozytywnych, z ograniczeniem do przedziału  $[0,5, c]$ :

$$w_k = \text{clip}\left(\frac{N_{\text{neg}}^{(k)}}{\max(N_{\text{pos}}^{(k)}, 1)}, 0,5, c\right), \quad (6)$$

gdzie  $c$  to górne ograniczenie (hiperparametr `pos_weight_cap`). Mechanizm ten zapobiega ignorowaniu rzadkich akcji (np. hamowania).

**Strata całkowita.** Ostateczna strata minimalizowana podczas treningu to średnia (5) po wszystkich  $K = 4$  etykietach i  $N$  przykładach w paczce:

$$\mathcal{L} = \frac{1}{N K} \sum_{i=1}^N \sum_{k=1}^K \ell_{w_k}(y_{ik}, z_{ik}). \quad (7)$$

## 5 Dane i ich przygotowanie

### 5.1 Charakterystyka zbioru

Surowy zbiór (`GameStatesTable.csv`) zawiera ponad 65 000 wierszy zarejestrowanych w trakcie rozgrywki. Każdy wiersz to jedna klatka gry: stan czujników oraz odpowiadające mu decyzje gracza.

### 5.2 Kroki przygotowania danych

Skrypt przygotowujący dane (zaimplementowany w funkcjach `load_data` oraz `build_features`) wykonuje następujące operacje:

1. wczytanie pliku CSV i oczyszczenie nazw kolumn,
2. konwersję kolumn na typ numeryczny i usunięcie wierszy z brakami (NaN), co chroni m.in. przed przypadkowym powtórzeniem nagłówka,
3. binaryzację etykiet do wartości  $\{0, 1\}$  progiem 0,5,
4. opcjonalne usunięcie duplikatów oraz początkowych „bezczynnych” klatek (gdy pojazd stoi i nie ma żadnego wejścia),
5. standaryzację cech numerycznych za pomocą `StandardScaler` (dopasowanego wyłącznie na zbiorze treningowym),
6. kodowanie one-hot typów promieni,
7. złączenie cech w macierz  $\mathbf{X}$  o 81 kolumnach oraz wektor etykiet  $\mathbf{y}$  o 4 kolumnach.

**Standaryzacja cech (z-score).** Cechy numeryczne mają różne zakresy (prędkość rzędu jednostek, odległości promieni rzędu setek), co utrudnia uczenie sieci. Standaryzacja sprowadza każdą cechę do średniej 0 i odchylenia standardowego 1:

$$x' = \frac{x - \mu}{\sigma}, \quad (8)$$

gdzie  $\mu$  i  $\sigma$  to średnia i odchylenie standardowe danej cechy. Co istotne, parametry te wyznacza się **wyłącznie na zbiorze treningowym**, a następnie tymi samymi wartościami przekształca zbiór walidacyjny — zapobiega to wyciekowi informacji ze zbioru testowego do procesu uczenia. Cechy katgoriczne (typ promienia) nie są skalowane, lecz kodowane binarnie (one-hot).

```

1 def build_features(df, scaler=None, fit_scaler=False):
2     numeric = df[NUMERIC_COLS].astype(np.float32).to_numpy()
3     if fit_scaler:
4         scaler = StandardScaler()
5         numeric =
6             scaler.fit_transform(numeric).astype(np.float32)
7     else:
8         numeric = scaler.transform(numeric).astype(np.float32)
9
10    type_features = []
11    for col in TYPE_COLS:
12        values = df[col].astype(int).to_numpy()
13        for ray_type in RAY_TYPE_VALUES:
14            #
15            kolejnosc: -1, 0, 1
16            one_hot = (values ==
17                ray_type).astype(np.float32).reshape(-1, 1)
18            type_features.append(one_hot)
19
20    x = np.concatenate([numeric] + type_features,
21        axis=1).astype(np.float32)
22    return x, scaler

```

Listing 3: Budowa cech wejściowych: standaryzacja liczb i one-hot typów (fragment `train_driver_model.py`).

### 5.3 Spójność uczenia i wdrożenia

Aby model wytrenowany w Pythonie działał identycznie w grze C++, parametry standaryzacji (średnia i odchylenie) są eksportowane do nagłówka C++ (`AIScalerData.h`). Dzięki temu funkcja `BuildAIInputFromGameState` buduje wektor wejściowy w dokładnie tej samej kolejności i skali co skrypt uczący:

```

1 static float ScaleNumericInput(float value, int index) {
2     return (value - AI_INPUT_MEAN[index]) /
3         AI_INPUT_SCALE[index];
4 }
5
6 // 1x car_speed + 20x ray_dist + 20x one-hot(ray_type) => 81
7     cech

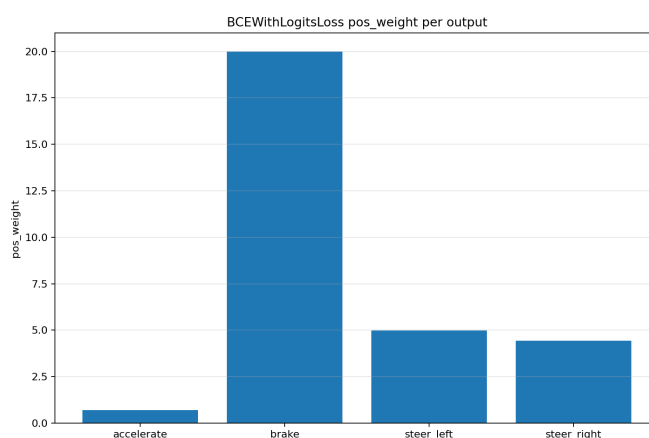
```

Listing 4: Standaryzacja cech po stronie gry (fragment `AIInputBuilder.cpp`).

## 6 Analiza eksploracyjna danych

### 6.1 Rozkład etykiet i nierównowaga klas

Analiza częstości wciśnięć klawiszy ujawnia silną **nierównowagę klas**. Przyspieszanie jest akcją dominującą, natomiast hamowanie pojawia się sporadycznie. Najlepiej obrazuje to wyznaczony wektor `pos_weight` (rysunek 1): waga dla hamowania osiąga górny limit (20), co oznacza, że bez korekty sieć ignorowałaby tę akcję. Skrety mają wagi rzędu 4–5, a przyspieszanie wagę poniżej 1 (klasa większościowa).



Rysunek 1: Wagi klasy pozytywnej (`pos_weight`) dla poszczególnych akcji. Bardzo wysoka waga hamowania odzwierciedla jego rzadkość w danych.

### 6.2 Wnioski z analizy zbalansowania

Z powodu nierównowagi klas **sama dokładność (accuracy) jest metryką mylącą**: model przewidujący zawsze „brak hamowania” osiągnąłby wysoką trafność dla tej etykiety, będąc bezużytecznym. Dlatego w ocenie modelu (rozdział 10) posługujemy się dodatkowo precyzją, czułością oraz miarą F1 liczonymi osobno dla każdej akcji, ze szczególnym uwzględnieniem hamowania. Z tego samego powodu w treningu zastosowano ważenie klas oraz strojenie progów decyzyjnych.

## 7 Architektura modelu

Wybrany model jest **wielowarstwowy perceptron (MLP)** zaimplementowany w bibliotece PyTorch (klasa `DriverNet`). Sieć przyjmuje 81 cech i zwraca 4 logity. W wariantach podstawowych wykorzystano normalizację wsadową (*batch normalization*), funkcje aktywacji ReLU oraz warstwę dropout przeciwdziałającą przeuczeniu.

```

1 class DriverNet(nn.Module):
2     def __init__(self, input_size: int, dropout: float):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(input_size, 128),
6             nn.BatchNorm1d(128),
7             nn.ReLU(),
8             nn.Dropout(dropout),
9             nn.Linear(128, 128),
10            nn.ReLU(),
11            nn.Dropout(dropout),
12            nn.Linear(128, 64),
13            nn.ReLU(),
14            nn.Linear(64, 4),      # 4 logity: acc, brake,
                                left, right
15        )
16    def forward(self, x):
17        return self.net(x)

```

Listing 5: Architektura sieci DriverNet z konfigurowalnym dropoutem (fragment `grid_search_driver_basic.py`).

Sieć zwraca logity (bez sigmoidy), ponieważ funkcja straty BCE WithLogitsLoss łączy w sobie sigmoidę i entropię krzyżową w sposób numerycznie stabilny. Sigmoida nakładana jest dopiero na etapie wnioskowania.

## 7.1 Teoria użytych mechanizmów

Architektura DriverNet składa się z kilku typów warstw, których działanie opisano poniżej.

**Warstwa liniowa (`nn.Linear`).** Realizuje przekształcenie afiniczne  $y = Wx + b$  (por. wzór (1)). Macierz wag  $W$  i wektor biasów  $b$  to parametry uczone. Same warstwy liniowe potrafią modelować tylko zależności liniowe, dlatego przeplata się je nieliniowymi aktywacjami.

**Aktywacja ReLU.** Funkcja *rectified linear unit* wprowadza nieliniowość:

$$\text{ReLU}(x) = \max(0, x). \quad (9)$$

Jest tania obliczeniowo, a jej gradient (równy 0 dla  $x < 0$  i 1 dla  $x > 0$ ) nie zanika dla dużych wartości dodatnich, co przyspiesza uczenie głębokich sieci.

**Normalizacja wsadowa (`BatchNorm1d`).** Dla każdej cechy normalizuje aktywacje w obrębie paczki danych, a następnie skaluje je uczonymi para-

metrami  $\gamma$  i  $\beta$ :

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta, \quad (10)$$

gdzie  $\mu_{\mathcal{B}}$ ,  $\sigma_{\mathcal{B}}^2$  to średnia i wariancja cechy w paczce, a  $\varepsilon$  to mała stała zapewniająca stabilność. Normalizacja stabilizuje rozkład aktywacji i pozwala na szybsze, stabilniejsze uczenie.

**Dropout.** Podczas treningu z prawdopodobieństwem  $p$  zeruje wyjścia poszczególnych neuronów, a pozostałe skaluje przez  $1/(1-p)$  (aby zachować wartość oczekiwaną):

$$\tilde{x}_j = \frac{m_j}{1-p} x_j, \quad m_j \sim \text{Bernoulli}(1-p). \quad (11)$$

Wymusza to redundancję reprezentacji i ogranicza współzależność neuronów, co przeciwdziała przeuczeniu. W trybie wnioskowania dropout jest wyłączony.

## 8 Metodyka eksperymentów

### 8.1 Podział danych i powtarzalność

Zbiór dzielony jest na część treningową i walidacyjną (domyślnie 80/20). Gdy w danych dostępna jest kolumna identyfikująca przejazd (`episode_id`), podział wykonywany jest *grupowo* (całe przejazdy trafiają do jednego zbioru), co zapobiega wyciekowi informacji między klatkami tego samego przejazdu i daje uczciwszą ocenę. Powtarzalność zapewnia ustalone ziarno losowości (`set_seed`) dla bibliotek `random`, `numpy` i `torch`.

### 8.2 Procedura treningu

Trening wykorzystuje optymalizator AdamW oraz mechanizm **wczesnego zatrzymania** (*early stopping*).

**Optymalizator AdamW.** Wagi aktualizowane są metodą gradientową z adaptacyjnym krokiem (Adam) i *odsprężoną* regularyzacją wag (*weight decay*). Dla gradientu  $\mathbf{g}_t$  utrzymywane są wykładnicze średnie pierwszego i drugiego momentu:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (12)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (13)$$

a aktualizacja parametrów ma postać:

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \left( \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} + \lambda \boldsymbol{\theta}_{t-1} \right), \quad (14)$$

gdzie  $\eta$  to współczynnik uczenia,  $\lambda$  to siła regularyzacji (**weight\_decay**), a  $\beta_1, \beta_2$  to współczynniki wygładzania momentów. Składnik  $\lambda \boldsymbol{\theta}_{t-1}$  „ściąga” wagi w stronę zera, ograniczając przeuczenie. W odróżnieniu od klasycznego Adama z regularyzacją L2, AdamW odprzega ten składnik od adaptacyjnego skalowania, co poprawia generalizację.

**Wczesne zatrzymanie.** Zapamiętywany jest stan modelu o najmniejszej stracie walidacyjnej, a uczenie przerywane jest, gdy strata nie poprawia się przez ustaloną liczbę epok (**patience**). Chroni to przed przeuczeniem i skraca czas treningu.

### 8.3 Metryki oceny

Ze względu na nierównowagę klas oraz wieloetykietowy charakter problemu raportowane są poniższe metryki. Oznaczmy przez TP, FP, FN, TN odpowiednio liczbę trafień prawdziwie/fałszywie dodatnich i fałszywie/prawdziwie ujemnych.

**Precyzja, czułość i F1.** Precyzja mówi, jaka część przewidzianych „wciśnięć” była trafna, a czułość (*recall*) — jaką część rzeczywistych wciśnięć udało się wykryć:

$$\text{Precyzja} = \frac{TP}{TP + FP}, \quad \text{Czułość} = \frac{TP}{TP + FN}. \quad (15)$$

Miara F1 to ich średnia harmoniczna, równoważąca oba aspekty — szczególnie przydatna przy nierównowadze klas:

$$F_1 = \frac{2 \cdot \text{Precyzja} \cdot \text{Czułość}}{\text{Precyzja} + \text{Czułość}}. \quad (16)$$

*Macro-F1* to średnia  $F_1$  po wszystkich  $K = 4$  akcjach (każda akcja waży tak samo, niezależnie od częstości):  $\text{macro-}F_1 = \frac{1}{K} \sum_k F_1^{(k)}$ .

**Dokładność dokładna.** Najsurowsza metryka — odsetek klatek, w których *wszystkie* cztery akcje przewidziano poprawnie jednocześnie:

$$\text{ExactAcc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{\mathbf{y}}_i = \mathbf{y}_i]. \quad (17)$$

**Pozostałe metryki.** Dodatkowo raportowane są: **strata walidacyjna** (`val_loss`), **dokładność per-przycisk** (dla każdej akcji osobno) oraz **PR-AUC** (pole pod krzywą precyzja-czułość, ang. *average precision*), będące dobrą miarą jakości dla bardzo rzadkich klas, takich jak hamowanie.

## 8.4 Logika decyzyjna i progi

Surowe prawdopodobieństwa zamieniane są na konkretne sterowanie z uwzględnieniem zasad gry: **hamowanie ma priorytet nad gazem**, a skręt to wybór *lewo albo prawo*, a nie obie naraz. Progi decyzyjne są osobnym, ważnym elementem rozwiązania — są strojone na zbiorze walidacyjnym pod kątem F1, co często poprawia jakość sterowania bardziej niż dalsze dostrajanie wag sieci.

```

1 def action_predictions(y_prob, thresholds):
2     pred = np.zeros_like(y_prob, dtype=np.int64)
3     t_acc, t_brake, t_left, t_right = thresholds.tolist()
4     brake_mask = y_prob[:, 1] >= t_brake
5     acc_mask = (~brake_mask) & (y_prob[:, 0] >= t_acc)    #
6                 hamulec > gaz
7     pred[brake_mask, 1] = 1
8     pred[acc_mask, 0] = 1
9     left_active = y_prob[:, 2] >= t_left
10    right_active = y_prob[:, 3] >= t_right
11    steer_active = left_active | right_active
12    pred[steer_active & (y_prob[:, 2] >= y_prob[:, 3]), 2] = 1
13    # lewo ALBO prawo
14    pred[steer_active & (y_prob[:, 3] > y_prob[:, 2]), 3] = 1
15    return pred

```

Listing 6: Zamiana prawdopodobieństw na sterowanie z priorytetem hamowania (fragment `grid_search_driver_basic.py`).

## 9 Model bazowy i strojenie konfiguracji

### 9.1 Model bazowy

Punktem odniesienia jest model wytrenowany w skrypcie `train_driver_model.py` z ustalonymi parametrami (`m.in. lr=0,001, weight_decay=10-4, batch_size=512, dropout 0,1, pos_weight_cap=20`, do 80 epok). Stanowi on bazę, względem której oceniamy zyski z optymalizacji.

### 9.2 Przeszukiwanie przestrzeni hiperparametrów

Globalną optymalizację zrealizowano metodą *grid search* (`grid_search_driver_basic.py`). Przeszukiwana siatka obejmuje parametry o największym wpływie na jakość kierowcy:

Tabela 1: Przestrzeń przeszukiwania hiperparametrów.

| Hiperparametr                                    | Testowane wartości            |
|--------------------------------------------------|-------------------------------|
| Współczynnik uczenia ( <code>lr</code> )         | $10^{-3}$ , $5 \cdot 10^{-4}$ |
| Regularyzacja ( <code>weight_decay</code> )      | $10^{-4}$ , $10^{-3}$         |
| Dropout                                          | 0,05, 0,10, 0,20              |
| Rozmiar wsadu ( <code>batch_size</code> )        | 128, 256                      |
| Limit wagi klasy ( <code>pos_weight_cap</code> ) | 5, 10, 20                     |

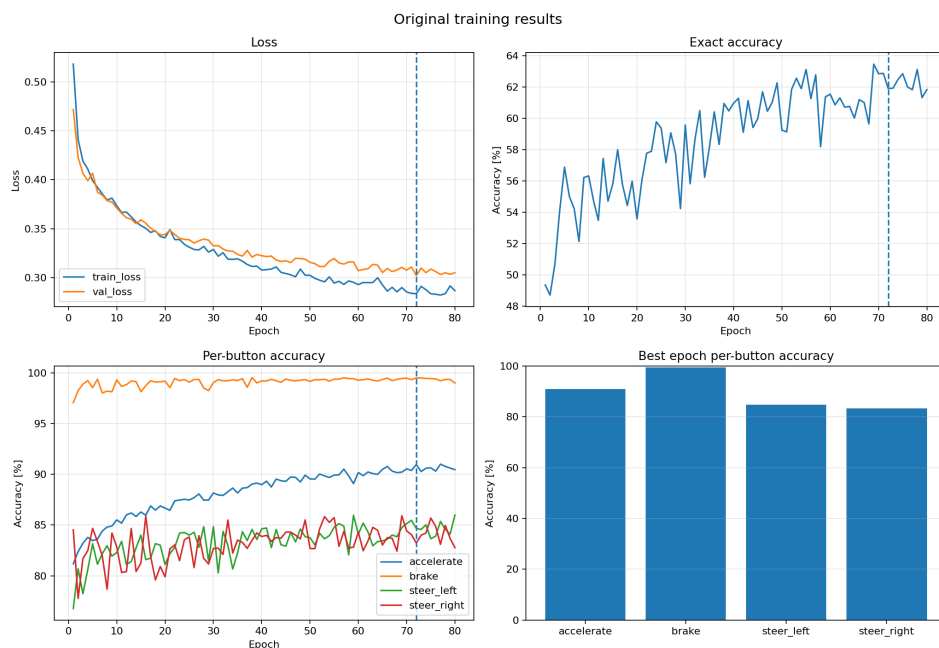
Dla każdej konfiguracji trenowany jest osobny model, a następnie *dodatkowo* strojone są progi decyzyjne (rozdział 8.4). Wyniki zapisywane są do pliku CSV oraz rankingów Markdown, co zapewnia powtarzalność i przejrzystość analizy. **Przypominamy jednak, że strojenie to jedynie etap pośredni:** ostatecznym kryterium jest jakość jazdy najlepszego znalezionego kierowcy.

## 10 Wyniki

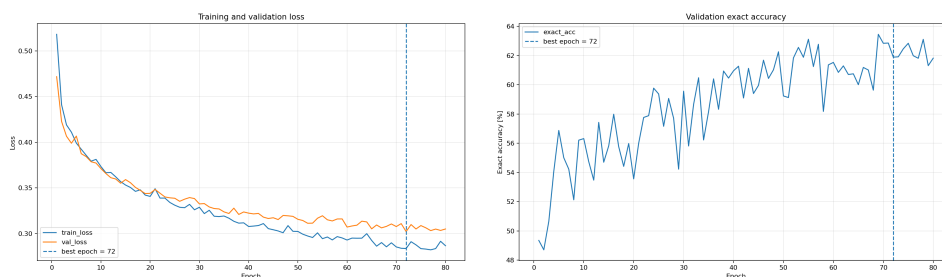
### 10.1 Przebieg uczenia modelu bazowego

Rysunek 2 zbiorczo przedstawia przebieg uczenia: spadek straty, wzrost dokładności dokładnej oraz dokładności poszczególnych akcji. Najlepszy stan modelu uzyskano w okolicy epoki 72.





Rysunek 2: Zbiorcze podsumowanie treningu modelu bazowego: strata, dokładność dokładna oraz dokładność per-przycisk wraz z wartościami w najlepszej epoce.



(a) Strata treningowa i walidacyjna.

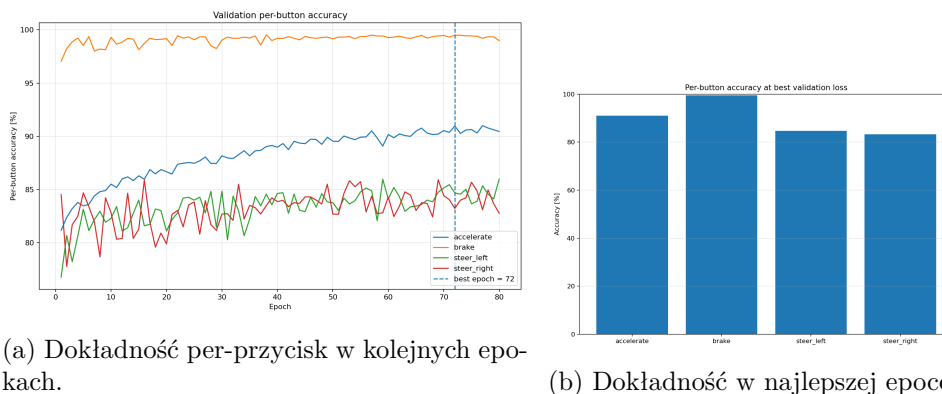
(b) Dokładność dokładna na walidacji.

Rysunek 3: Krzywe uczenia. Strata walidacyjna stabilizuje się ok. 0,30, a dokładność dokładna rośnie do ok. 62–63%. Niewielka różnica między stratą treningową a walidacyjną świadczy o umiarkowanym przeuczeniu.

## 10.2 Jakość poszczególnych akcji

Dokładność per-przycisk (rysunki 4a i 4b) pokazuje, że model najlepiej radzi sobie z hamowaniem (ok. 99% — tu jednak decyduje rzadkość klasy) oraz przyspieszaniem (ok. 91%). Trudniejsze okazują się skrety (ok. 83–85%), co jest zgodne z intuicją: decyzja o skręcie wymaga subtelniejszej interpretacji

układu promieni niż decyzja o jeździe na wprost.



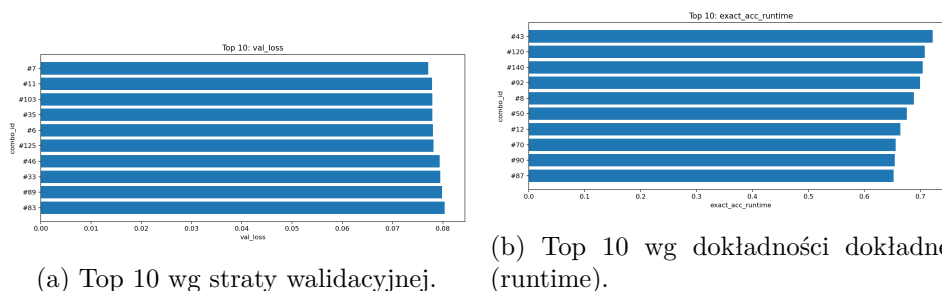
(a) Dokładność per-przycisk w kolejnych epokach.

(b) Dokładność w najlepszej epoce.

Rysunek 4: Dokładność dla poszczególnych akcji. Skręt w lewo i w prawo to akcje najtrudniejsze do nauczenia.

### 10.3 Wyniki strojenia hiperparametrów

Ranking konfiguracji według straty walidacyjnej (rysunek 5a) pokazuje, że wiele konfiguracji daje bardzo zbliżone wyniki ( $val\_loss$  ok. 0,077–0,080), co świadczy o stabilności architektury względem testowanych hiperparametrów. Ranking według dokładności dokładnej w trybie runtime (rysunek 5b) wskazuje najlepsze konfiguracje przekraczające 70% poprawnie odtworzonych pełnych zestawów akcji.

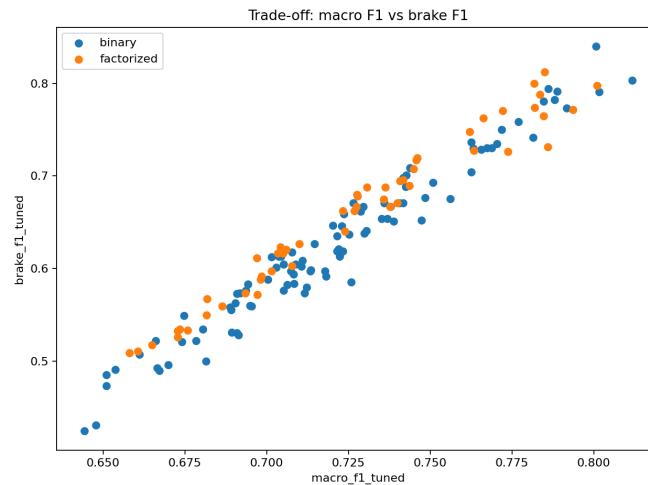


(a) Top 10 wg straty walidacyjnej.

(b) Top 10 wg dokładności dokładnej (runtime).

Rysunek 5: Rankingi konfiguracji z przeszukiwania siatki hiperparametrów.

Rysunek 6 ilustruje zależność między uśrednioną miarą F1 (macro-F1) a miarą F1 dla samego hamowania. Widać silną dodatnią korelację — konfiguracje dobre „ogólnie” są zwykle dobre także w trudnej akcji hamowania, choć dla najlepszych modeli pojawia się pewien kompromis między tymi metrykami.



Rysunek 6: Kompromis między macro-F1 a F1 dla hamowania (po strojeniu progów). Każdy punkt to jedna konfiguracja modelu.

#### 10.4 Porównanie z modelem bazowym

Najlepsza konfiguracja wyłoniona w przeszukiwaniu osiąga niższą stratę walidacyjną i wyższe macro-F1 niż model bazowy, przy zachowaniu wysokiej skuteczności hamowania. Zysk wynika w dużej mierze z **łącnego** dostrojenia wag sieci oraz progów decyzyjnych, a nie z pojedynczego hiperparametru — co potwierdza, że celem jest sprawny kierowca, a nie optymalizacja jednej liczby.

### 11 Wdrożenie modelu w grze

Wytrenowany model eksportowany jest do formatu **ONNX** i ładowany w grze przez klasę **NeuralDriver** (biblioteka ONNX Runtime). W każdej klatce gra:

1. buduje 81-elementowy wektor wejściowy ze stanu pojazdu (**BuildAIInputFromGameState**),
2. uruchamia model i otrzymuje 4 logity,
3. nakłada sigmoideę, uzyskując prawdopodobieństwa,
4. stosuje progi i zasady decyzyjne (priorytet hamowania, wybór jednego kierunku skrętu), zamieniając prawdopodobieństwa na sterowanie.

```

1 Controls NeuralDriver::PredictControls(const GameState& state)
2 {
3     std::vector<float> probs = PredictProbabilities(state);
4     float pAccelerate = probs[0], pBrake = probs[1];
5     float pSteerLeft = probs[2], pSteerRight = probs[3];
6
7     Controls controls{};
8     if (pBrake > 0.50f) controls.brake = 1.0f;
9         // priorytet hamulca
10    else if (pAccelerate > 0.30f) controls.accelerate = 1.0f;
11
12    float steeringThreshold = 0.35f;
13    if (pSteerLeft > steeringThreshold || pSteerRight >
14        steeringThreshold) {
15        if (pSteerLeft > pSteerRight) controls.steerLeft =
16            1.0f; // lewo ALBO prawo
17        else controls.steerRight =
18            1.0f;
19    }
20    return controls;
21 }

```

Listing 7: Zamiana prawdopodobieństw na sterowanie w grze (fragment NeuralDriver.cpp).

Przełączanie między kierowcą-człowiekiem a kierowcą-AI odbywa się w grze klawiszem M. Dzięki spójnemu budowaniu wejścia i identycznym progom decyzyjnym po obu stronach (Python i C++) zachowanie modelu w grze odpowiada wynikom z walidacji.

## 12 Analiza błędów i ograniczenia

- **Najczęściej mylone akcje:** skręt w lewo i skręt w prawo. Model bywa niezdecydowany w sytuacjach symetrycznych, gdy oba kierunki mają zbliżone prawdopodobieństwo — stąd reguła wyboru jednego (silniejszego) kierunku.
- **Imitacja, nie optymalizacja jazdy:** model uczy się odwzorowywać decyzje gracza, więc dziedziczy jego błędy i nie eksploruje lepszych strategii niewystępujących w danych.
- **Zależność od jakości danych:** sytuacje rzadkie na torze (ostre zakręty, cofanie) są słabo reprezentowane, co obniża skuteczność modelu w takich momentach.
- **Wyciek informacji:** przy losowym podziale na poziomie pojedynczych klatek kolejne, niemal identyczne klatki tego samego przejazdu mogą trafić do treningu i walidacji jednocześnie. Podział grupowy po przejazdach ogranicza ten efekt.

## 13 Wnioski i kierunki dalszych prac

### Wnioski.

- Udało się **nauczyć komputer sterowania samochodem** w grze 2D — sieć neuronowa na podstawie 20 promieni i prędkości podejmuje sensowne decyzje sterujące w czasie rzeczywistym.
- Kluczowe okazały się: ważenie rzadkich klas (`pos_weight`) oraz strojenie progów decyzyjnych, a nie samo dostrajanie wag sieci.
- Najtrudniejszym elementem jazdy są skrety; hamowanie, mimo rzadkości, jest dobrze rozpoznawane dzięki ważeniu klas.
- Przeszukiwanie hiperparametrów potraktowano jako narzędzie poprawy kierowcy — różnice między najlepszymi konfiguracjami są niewielkie, co świadczy o stabilności rozwiązania.

### Dalsze prace.

- zastosowanie **uczenia ze wzmocnieniem** (RL), aby model optymalizował czas przejazdu, a nie jedynie naśladował gracza,
- dodanie informacji o **historii** (kilka poprzednich klatek lub sieć rekurencyjna) dla płynniejszego sterowania,
- zbieranie danych z wielu torów i wielu graczy w celu lepszej generalizacji,
- rejestrowanie identyfikatora przejazdu (`episode_id`) dla rzetelnego podziału grupowego i uczciwszej oceny.

## Literatura

- [1] A. Paszke i in., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [2] F. Pedregosa i in., *Scikit-learn: Machine Learning in Python*, JMLR, 2011.
- [3] ONNX Runtime, *Open Neural Network Exchange Runtime*, <https://onnxruntime.ai>.
- [4] R. Santamaria, *raylib — a simple and easy-to-use library to enjoy videogames programming*, <https://www.raylib.com>.